

Security Audit

Blocknite (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Program Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	29
Conclusion	30
Our Methodology	31
Disclaimers	33
About Hashlock	34

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Blocknite team partnered with Hashlock to conduct a security audit of their Solana program. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Blocknite is an idle play-to-earn RPG built on the Solana blockchain, where players assume the role of Altarin lords commanding armies of Treanin champions to defend the realm of Trea from the invading Orgurin horde. The game combines traditional RPG mechanics with Web3 elements, allowing players to forge Soul Stones, summon Treanin NFTs, and engage in strategic battles to earn the native \$NITE token. With low time commitment—requiring players to check in every 8 hours—Blocknite offers a growing ecosystem that includes features like boss fights, co-op PvE, and farming. The game emphasizes true ownership of in-game assets, enabling players to buy, sell, and trade Treanin warriors in a decentralized marketplace, all while participating in a dynamic and evolving storyline.

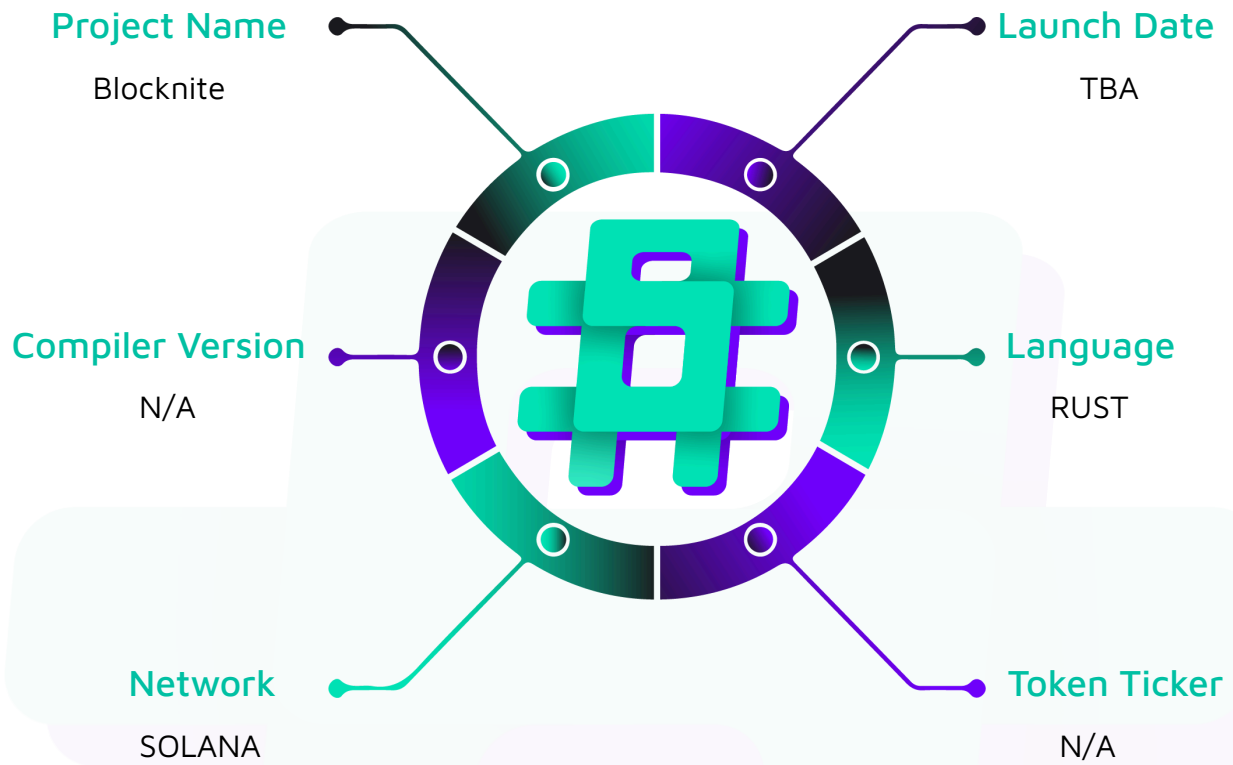
Project Name: Blocknite

Project Type: DeFi, Presale

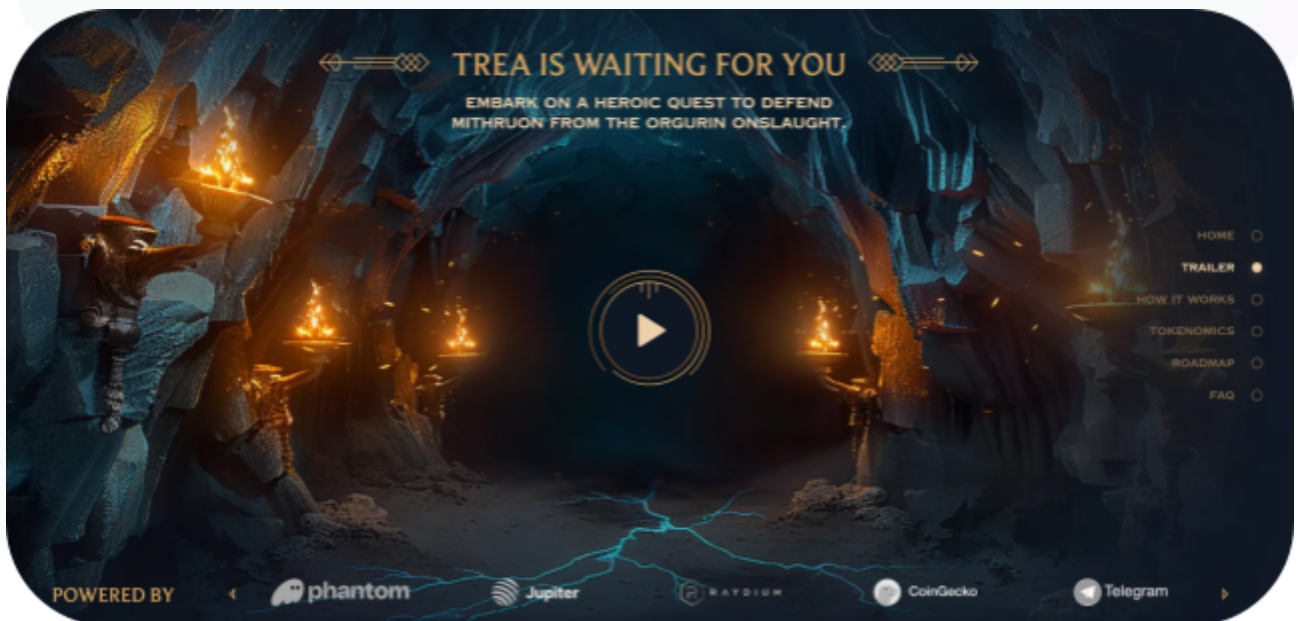
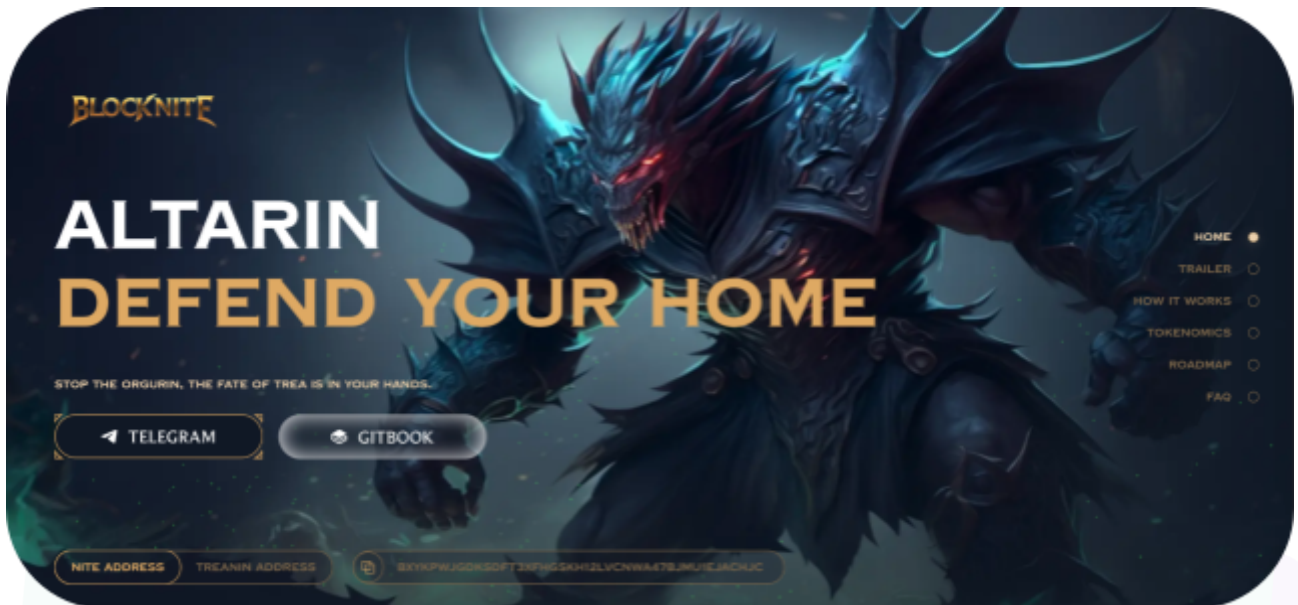
Website: <https://blocknite.io/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Solana code within the Blocknite project, the scope of work included a comprehensive review of the programs listed below. We tested the Solana program to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Blocknite Programs
Platform	Solana / Rust
Audit Date	August, 2025
Program 1	blocknite-presale
Audited GitHub Commit Hash	845d71fed93f4ad91c4111878946f152311c7854
Fix Review GitHub Commit Hash	c9fafa27a4ff1f8e6aa22b840c79decdf4c82f31

Security Rating

After Hashlock's Audit, we found the program to be **"Secure"**. The program all follows standard logic, with correct ordering.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Program Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

7 High severity vulnerabilities

5 Medium severity vulnerabilities

3 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Program Functions

Claimed Behaviour	Actual Behaviour
blocknite_presale - Initialize global config with admin stablecoin mints, Pyth price account, and staleness threshold - Create a presale with price caps schedule, and bookkeeping keyed by a counter - Update presale parameters, including price caps start_time, and end_time - Set the reward token mint and its decimals - Deposit reward tokens to finalize the sale and enable claims - Purchase allocation with SOL using the Pyth SOL USD price - Purchase allocation with USDC or USDT via TransferChecked - Claim vested rewards based on TGE and linear vesting - Admin withdraws collected SOL and stablecoins	The program achieves this functionality.

Code Quality

This audit scope involves the Solana program of the Blocknite project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is well commented on and closely follows best practice nat-spec styling. Some comments are not correctly aligned with code functionality.

Audit Resources

We were given the Blocknite project Solana program code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this program infrastructure are based on well-known industry standard open source projects.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] blocknite-presale#purchase_with_sol - User supplied Pyth account combined with `get_price_unchecked` enables price spoofing

Description

The purchase flow accepts an arbitrary account as the Pyth price source and then reads price data without verifying program ownership or signatures. A malicious caller can forge a price update to buy out the sale for negligible SOL.

Vulnerability Details

The `Accounts` struct for purchases takes `pyth_price_account` as an unchecked account. In the code path where a non-default address is supplied, the program forwards this account to the oracle helper with a fixed staleness window. Inside the oracle, the code logs the owner but does not assert it. It parses arbitrary bytes into a `PriceUpdateV2` and calls `get_price_unchecked`. No CPI to the Pyth receiver is performed. This allows a fabricated account to pass through as a fresh price source.

Impact

Attackers can set an arbitrarily high SOL/USD rate and purchase effectively the entire remaining cap for dust. If reward tokens are later deposited, they can claim vastly more tokens than paid for..

Recommendation

We recommend binding `pyth_price_account` to the configured address and verify the account is owned by the official Pyth receiver program. Use the Pyth Receiver SDK verification flow in-transaction instead of parsing bytes and using `get_price_unchecked`.

Status

Resolved

[H-02] blocknite-presale#tokens_purchased - Unit mismatch under-delivers by 10^{dec} and strands surplus in the vault

Description

Purchased amounts are stored as whole tokens, while claims and deposits are handled in base units. This creates a 10^{dec} decimals mismatch that underpays users and leaves unclaimed surplus.

Vulnerability Details

`tokens_purchased` is computed as USD divided by `token_price_usd`, both scaled to 6 decimals. No scaling to the reward mint's decimals is applied. Claims transfer base units with `TransferChecked` and the mint decimals.

Separately, the deposit path calculates the required tokens by multiplying by 10^{dec} , which the vault receives in base units. Users' stored entitlements, therefore, lack the mint-decimals factor and will never match the deposited base units.

Impact

Users receive roughly 10^{dec} fewer tokens than paid for. The vault holds the "extra" base units, and the difference becomes stuck.

Recommendation

We recommend storing or computing `tokens_purchased` in base units by multiplying the whole-token figure by $10^{\text{reward_decimals}}$. Align both SOL and USD purchase paths and update claim math to keep all amounts in the same unit.

Status

Resolved

[H-03] blocknite-presale#purchase_with_sol - Devnet fallback backdoor bypasses the oracle entirely

Description

Passing the default pubkey selects a fixed USD price path meant for devnet and bypasses Pyth.

Vulnerability Details

When `pyth_price_account.key` equals `Pubkey::default`, the handler branches into a test path that sets a constant price like `100_000_000` in micro USD and uses it to compute `usd_amount`. That branch skips `get_sol_price_usd` and therefore avoids freshness checks and feed identity validation.

There is no guard on `global_config` to disable this in production, and no `require!` that restricts the fallback to test deployments. Any caller can therefore opt out of the oracle by passing the default key and force the hard-coded price into the sale math.

Impact

An attacker can deliberately trigger the fallback to fill the cap cheaply whenever the real market price is higher than the hard-coded value.

Recommendation

We recommend removing the fallback or gate it behind a configuration flag that is only enabled on non-mainnet deployments and only by the admin.

Status

Resolved

[H-04] blocknite-presale#pyth_price_update_account - Configured feed is stored but never enforced during purchase

Description

Even if the admin sets the Pyth update account in the global config, purchases accept any account.

Vulnerability Details

The `initialize_config` writes `global_config.pyth_price_update_account` and `global_config.price_staleness_threshold` but `purchase_with_sol` never compares `ctx.accounts.pyth_price_account.key` to this stored key. The handler also does not assert `pyth_price_account.owner` is the Pyth Receiver program ID, which means even a correctly configured feed can be bypassed.

The unchecked account is passed to `sol_to_usd_with_oracle`, where `get_price_unchecked` is ultimately invoked. Configuration, therefore, becomes advisory rather than enforced, and the purchase path remains vulnerable to arbitrary account injection.

Impact

Facilitates the spoofed-price attack described in previous issues.

Recommendation

We recommend requiring `pyth_price_account` to equal `global_config.pyth_price_update_account` and verifying the owner to be the official Pyth receiver. Then verify signatures via CPI.

Status

Resolved

[H-05] blocknite-presale#get_sol_price_usd - No owner or signature verification

Description

The oracle helper relies on parsing arbitrary account data and calling `get_price_unchecked` without verifying authenticity.

Vulnerability Details

The `get_sol_price_usd` and its wrapper `sol_to_usd_with_oracle` deserialize a `PriceUpdateV2` from `pyth_price_account.data` and then read fields like `price` and `publish_time` using `get_price_unchecked`. The only validation is a staleness comparison against `now`, which uses `publish_time` from untrusted data.

There is no assertion that `AccountInfo.owner` equals the Pyth Receiver program ID and no CPI through the Pyth verification interface to validate guardian signatures and feed ID. This means any buffer with the expected layout and a recent `publish_time` will be accepted as a fresh price source.

Impact

Untrusted or replayed price data can pass as valid as long as the timestamp is recent.

Recommendation

We recommend enforcing program ownership and using the Pyth Receiver CPI verification to validate signatures and feed IDs before accepting prices.

Status

Resolved

[H-06] blocknite-presale#user_participation - Bump not initialized causes PDA seed mismatch at claim

Description

The user participation account is created with `init_if_needed`, but the bump is never written to the state. Claim uses the bump from account data in its seeds constraint.

Vulnerability Details

The `UserParticipation` PDA is created with `init_if_needed` under seeds such as `["user_participation", presale.key(), user.key()]` with a runtime bump stored in `ctx.bumps["user_participation"]`. The program never writes this bump into `UserParticipation.bump`.

Later, the `claim_tokens` context enforces seeds using `bump = user_participation.bump`, which remains zero for first-time participants. Anchor derives a PDA with the zero bump that does not match the originally created address, and the seeds constraint fails during the CPI that performs the token transfer.

Impact

Every claim reverts to users whose participation account was first created by the program.

Recommendation

On first creation, we recommend setting `user_participation.bump` from `ctx.bumps`. Alternatively, remove the bump from the on-chain state and use a plain bump in constraints.

Status

Resolved

[H-07] blocknite-presale#user_participation - purchase_with_usd misinterprets amounts when stablecoin mint decimals are not 6

Description

The presale state and comments clearly assume “USD” quantities are tracked with 6 decimals for both the cap and the price per token. However, in `purchase_with_usd` the code adds the raw `amount` directly into `total_usd_raised` and per-mint subtotals without first normalizing it to 6 decimals. The only place mint decimals are read is for `TransferChecked`, so accounting can silently drift if `spl_token_mint.decimals` is not exactly 6.

Vulnerability Details

The `amount` is used as “USD with 6 decimals” when computing `new_total` and `tokens_purchased`, and it is also added to `total_usdc_raised` or `total_usdt_raised` directly. If a mint has 8 or 9 decimals, then one on-chain “unit” is not a micro-USD, so the cap, price, and contribution math diverge.

An attacker that can point config to a stablecoin clone with different decimals can over- or under-count contributions relative to the declared 6-dec accounting, while transfers still succeed thanks to `TransferChecked` using the mint’s true decimals.

Impact

Contributions may be mis-accounted, and the presale cap enforced at the wrong effective level. Users can overpay relative to the token price scale, or the sale can “overfill” without the state reflecting it.

Recommendation

We recommend enforcing `spl_token_mint.decimals == 6` for the configured USDC and USDT mints, or normalize all incoming `amount` values to 6 decimals before applying cap and price arithmetic. Also store and enforce the expected decimals alongside each configured mint.

Status

Resolved



Medium

[M-01] blocknite-presale#withdraw_sol_funds - Admin can withdraw collected funds before sale finalization

Description

The SOL and USD withdrawal functions are gated only by admin authority and available balance. They do not require the presale to have ended or been finalized.

Vulnerability Details

The `withdraw_sol_funds` transfers `current_balance - rent_exempt_minimum` to `admin` after checking only `global_config.admin`. The `withdraw_usd_funds` moves the full `presale_spl_vault.amount` to `admin_spl_token_account` under the same minimal check.

Neither function checks `presale.is_finalized` nor calls a helper like `presale.has_ended_safe` to gate withdrawals until the sale completes. This lets an authorized admin empty `sol_vault` or `spl_vault` while `claiming_enabled` is `false`, and while users still expect escrow.

Impact

Funds can be removed mid-sale. Buyers rely on off-chain trust rather than on-chain guarantees.

Recommendation

We recommend requiring `presale.has_ended_safe` and `presale.is_finalized` before allowing withdrawals. Consider an explicit finalization flag that gates these operations.

Status

Partially resolved. Withdrawals now require the presale to have ended, but there's still no `is_finalized` guard, so funds can be withdrawn right after the sale ends but before reward tokens are deposited.

[M-02] blocknite-presale - Hard-coded 60-second staleness ignores configured threshold

Description

The purchase path passes a fixed 60-second freshness window to the oracle, while config exposes a default of five minutes and a per-deployment setting.

Vulnerability Details

Although `DEFAULT_PRICE_STALENESS_THRESHOLD` is set to 300 and `initialize_config` writes `global_config.price_staleness_threshold`, the call to `sol_to_usd_with_oracle` passes a literal 60. The freshness check inside the oracle compares `now - price.publish_time` against this argument rather than reading `global_config.price_staleness_threshold`.

Operators may believe they configured a five-minute window while the executable enforces one minute, which changes acceptance of updates during latency spikes and incident response.

Impact

Unexpected rejections or acceptance of forged prices within 60 seconds. Operators may assume a different policy than what the chain enforces.

Recommendation

We recommend using `global_config.price_staleness_threshold` when invoking oracle conversions.

Status

Resolved

[M-03] blocknite-presale - Missing `token_program` in two places risks deriving under the wrong program

Description

Two associated token constraints omit the token program. In Token-2022 contexts, these ATAs may resolve under the wrong program.

Vulnerability Details

In the `PurchaseWithUsd` accounts struct, the `user_token_ata` constraint omits `associated_token::token_program = token_program`. In `WithdrawUsdFunds` the `admin_spl_token_account` constraint has the same omission.

When the mint uses Token 2022, the ATA derivation must include the token program ID, or it will default to the legacy SPL Token program. The resulting `associated_token::authority` and `associated_token::mint` tuple produces an address that does not exist under Token 2022, which leads to erroneous transfers or missing account errors.

Impact

Purchases and withdrawals can fail or point to a different ATA than expected.

Recommendation

We recommend adding `associated_token::token_program = token_program` to both constraints to align with Token-2022 behavior.

Status

Resolved

[M-04] blocknite-presale#UpdatePresale - “No end time” allowed via update but disallowed at creation

Description

Creation enforces `start_time < end_time` while update sets `end_time` unconditionally, even if zero, as long as certain checks pass. This breaks invariants and allows indefinite fundraising after creation.

Vulnerability Details

The `create_presale` enforces the `require!(params.start_time < params.end_time, PresaleError::InvalidTimeParameters)` and rejects `end_time == 0`. The `update_presale` allows `presale.end_time = 0` because it only checks the inequality when `new_end_time > 0` then assigns the value.

This breaks the invariant that every sale has a finite interval unless explicitly flagged elsewhere and impacts helpers like `has_ended_safe` that expect a sensible end bound. Off-chain dashboards will show an inconsistent status when `end_time` is zero after an update, but would have been rejected at creation.

Impact

Admins can bypass the creation guard by updating to a no-end presale unexpectedly.

Recommendation

We recommend aligning both paths. Either support `end_time == 0` consistently or prohibit it in updates as well.

Status

Resolved

[M-05] blocknite-presale#DepositRewardTokens - Re-entrant finalization lets claim start be reset and extra tokens be deposited

Description

The deposit step sets `is_finalized` and `claim_start_time`, but does not prevent re-execution. A subsequent call can reset the vesting start and over-deposit.

Vulnerability Details

The `deposit_reward_tokens` transfers `required_amount` into `reward_vault`, then sets `presale.claiming_enabled = true`, `presale.is_finalized = true` and `presale.claim_start_time = clock::get()?.unix_timestamp`. There is no guard like `require(!presale.is_finalized, Error)` to prevent a second call.

A later invocation overwrites `claim_start_time`, which shifts vesting cliffs and also moves additional tokens into the vault that the entitlement math never anticipates. The function does not check that `reward_vault.amount` already equals `required_amount` so idempotency is not enforced.

Impact

Accidental or malicious re-deposit shifts the vesting start and can trap extra tokens.

Recommendation

We recommend requiring the `!presale.is_finalized` before proceeding. Optionally add an idempotency check on the vault balance against the required amount.

Status

Resolved

Low

[L-01] blocknite-presale - Integer division causes silent rounding loss

Description

The `sol_to_usd_with_oracle` converts lamports to micro USD using integer math that divides by `LAMPORTS_PER_SOL * 100`, which floors fractional cents. The purchase path then sets `tokens_purchased = usd_amount / token_price_usd`, which again truncates.

In `claim_tokens` the vesting logic computes `tge_amount` and `linear_amount` using integer division by `PERCENTAGE_SCALE` and by the vesting duration, which discards remainders.

Without tracking a `remainder_usd` or `remainder_tokens` per user, these truncation errors accumulate as unclaimable dust in the reward vault.

Users receive slightly fewer tokens than expected, and dust accumulates in vaults.

Recommendation

We recommend tracking and applying fractional remainders per user or adopting consistent rounding that matches distribution math. Provide a dust sweep after vesting ends.

Status

Resolved

[L-02] blocknite-presale#WithdrawSolFunds - Misleading log prints the full presale balance rather than the net withdrawal

Description

The `withdraw_sol_funds` logs `presale_lamports` as withdrawn even though the transfer uses `withdrawable_sol = current_balance - rent_exempt_minimum`. The message, therefore, overstates the outflow relative to the actual transfer amount.

This creates confusion in monitoring and forensic analysis when reconciling logs with on-chain balance deltas.

Recommendation

We recommend logging the computed `withdrawable_sol` so the message matches the transfer.

Status

Resolved

[L-03] blocknite-presale - Emergency pause flag exists but cannot be toggled on chain

Description

The GlobalConfig includes `emergency_pause: bool` that is checked in `purchase_with_sol` and `purchase_with_usd` via `require(!global_config.emergency_pause, PresaleError::EmergencyPause)`. The only place that sets the field is `initialize_config`, which writes `false`.

There is no admin instruction, such as `set_emergency_pause` to flip the flag at runtime. This makes the program non-pausable.

Recommendation

We recommend adding an admin-only toggle instruction with appropriate access control.

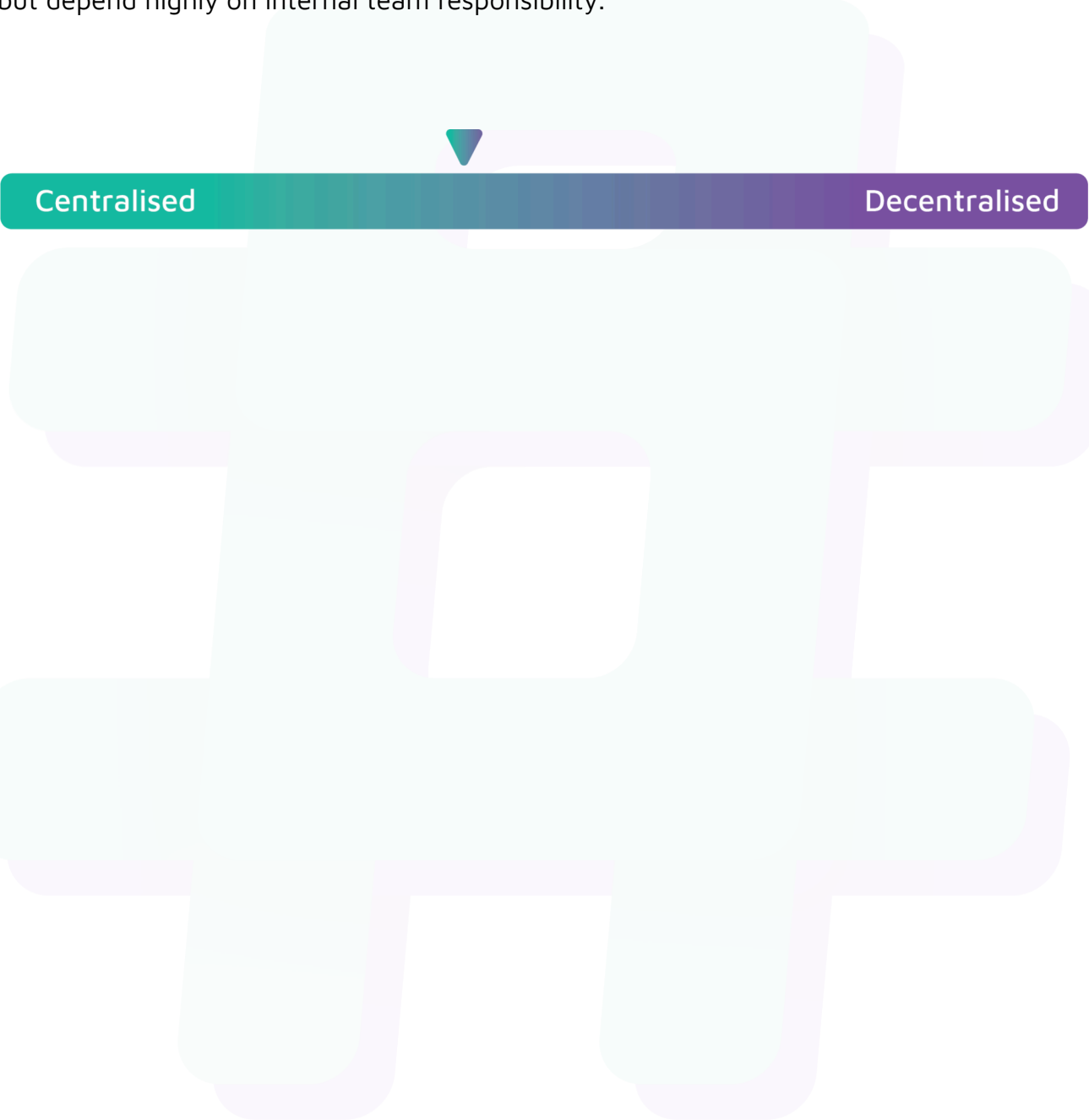
Status

Resolved

Centralisation

The Blocknite project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Blocknite project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.